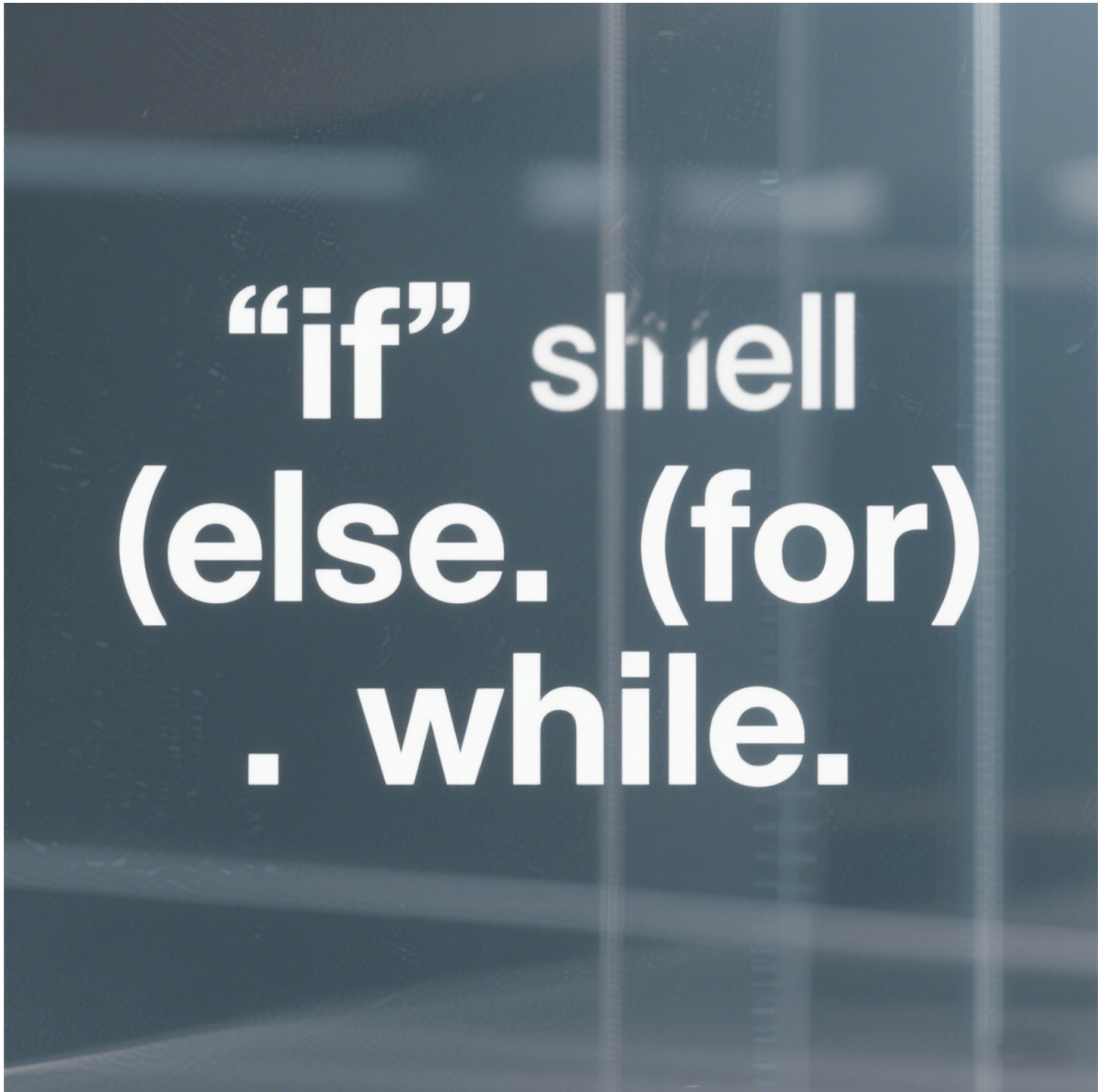


shell编程



一、shell脚本的输入

如果shell脚本中的变量随时都会变化，那么我们通常需要向shell脚本传入随时可以变动的参数作为变量的值

(1.1) 位置参数

即 `$n`、`$*`、`$@`、`$#` 这四个特殊符号

位置参数可以与 `shift` 左移指令结合使用

`shift`在shell脚本中，每执行一次，参数序列顺次左移一个位置，`$#`的值减1，用于分别处理每个参数，移出去的参数，不再可用

范例

```
#!/bin/bash
echo "现在$1的值是 $1"
echo "执行shift"
shift
echo "执行shift后的$1的值是 $1"
```

执行结果

```
[root@theon shell_class]# sh shift.sh aaa bbb
现在$1的值是 aaa
执行shift
执行shift后的$1的值是 bbb
```

利用shift我们可以方便的进行位置参数操作，比如求和等操作

(1.2) 键盘输入

使用 `read` 命令，从键盘向脚本输入参数

```
read [选项] [变量名]
```

选项：

- p "提示信息"：在等待read输入时，输出提示信息
- t 秒数：read命令会一直等待用户输入，使用此选项可以指定等待时间
- n 字符数：read命令只接受指定的字符数，就会执行
- s：隐藏输入的数据，适用于机密信息的输入

范例如下脚本：

```
#!/bin/bash
read -p "input your name: " name
echo "hello ${name} nice to meet you"
```

执行效果

```
[root@theon shell_class]# sh read.sh
input your name:
```

输入参数

```
input your name: Json
hello Json nice to meet you
[root@theon shell_class]#
```

(1.3) 管道符输入

输入时需要注意，由于shell脚本执行需要的是参数，因此要添加 `xargs` 将管道内文本类型信息转为参数信息

我们以之前shift的范例为例子：

```
[root@theon shell_class]# cat a.txt
aaa
bbb
[root@theon shell_class]# cat a.txt |xargs sh shift.sh
现在$1的值是 aaa
执行shift
执行shift后的$1的值是 bbb
[root@theon shell_class]#
```

二、shell编程语句

(2.1) 判断语句

2.1.1 判断类型和判断表达式

- 1. 文件类型判断表达式

如下表，蓝色为常用的判断

测试选项	作 用
-b 文件	判断该文件是否存在，并且是否为块设备文件（是块设备文件为真）
-c 文件	判断该文件是否存在，并且是否为字符设备文件（是字符设备文件为真）
-d 文件	判断该文件是否存在，并且是否为目录文件（是目录为真）
-e 文件	判断该文件是否存在（存在为真）
-f 文件	判断该文件是否存在，并且是否为普通文件（是普通文件为真）
-L 文件	判断该文件是否存在，并且是否为符号链接文件（是符号链接文件为真）
-p 文件	判断该文件是否存在，并且是否为管道文件（是管道文件为真）
-s 文件	判断该文件是否存在，并且是否为非空（非空为真）
-S 文件	判断该文件是否存在，并且是否为套接字文件（是套接字文件为真）

判断表达式如下范例：

```
test -e /etc/passwd # /etc/passwd存在则为真
[ -e /etc/passwd ] # /etc/passwd存在则为真，注意 [ ]前后有空格
# 通常的用法
test -e /etc/passwd && echo ok || echo no
[ -e /etc/passwd ] && echo ok || echo no
```

- 2. 文件权限判断表达式

如下表，蓝色为常用

测试选项	作 用
<u>-r 文件</u>	判断该文件是否存在，并且是否该文件拥有 <u>读权限</u> （有读权限为真）
<u>-w 文件</u>	判断该文件是否存在，并且是否该文件拥有写权限（有写权限为真）
<u>-x 文件</u>	判断该文件是否存在，并且是否该文件拥有执行权限（有执行权限为真）
-u 文件	判断该文件是否存在，并且是否该文件拥有SUID权限（有SUID权限为真）
-g 文件	判断该文件是否存在，并且是否该文件拥有SGID权限（有SGID权限为真）
-k 文件	判断该文件是否存在，并且是否该文件拥有SBit权限（有SBit权限为真）

- 3.文件比较判断表达式

如下表

测试选项	作 用
<u>文件1 -nt 文件2</u>	判断文件1的 <u>修改时间</u> 是否比文件2的 <u>新</u> （如果新则为真）
文件1 <u>-ot</u> 文件2	判断文件1的修改时间是否比文件2的 <u>旧</u> （如果旧则为真）
文件1 <u>-ef</u> 文件2	判断文件1是否和文件2的Inode号一致，可以理解为两个文件是否为同一个文件。这个判断用于判断硬链接是很好的方法

- 4.整数比较判断

如下表

测试选项	作 用
整数1 <u>-eq</u> 整数2	判断整数1是否和整数2相等（相等为真）
整数1 <u>-ne</u> 整数2	判断整数1是否和整数2不相等（不相等位置）
整数1 <u>-gt</u> 整数2	判断整数1是否大于整数2（大于为真）
整数1 <u>-lt</u> 整数2	判断整数1是否小于整数2（小于位置）
整数1 <u>-ge</u> 整数2	判断整数1是否大于等于整数2（大于等于为真）
整数1 <u>-le</u> 整数2	判断整数1是否小于等于整数2（小于等于为真）

补充：由于 eq、lt等缩写不容易记住，因此通常不使用这种方式进行整数比较判断，而是结合 (()) 和运算符进行判断

范例：

```
# 使用运算缩写
[ 2 -lt 3 ] && echo ok || echo no
# 使用运算符
((2 < 3)) && echo ok || echo no
```

- 5.字符串对比判断表达式

字符串判断十分常用，如下表：

测试选项	作 用
-z 字符串	判断字符串是否为空（为空返回真）
-n 字符串	判断字符串是否为非空（非空返回真）
字符串1 == 字符串2	判断字符串1是否和字符串2相等（相等返回真）
字符串1 != 字符串2	判断字符串1是否和字符串2不相等（不相等返回真）

范例：

```
name="lucy"
[ -n ${name} ] && echo ok || echo no
# name变量不为空，输出为ok
```

2.2.2 流程判断

- if语句

单分支if语句，使用方式如下：

```
if [ 条件判断式 ]; then
    程序
fi
或者
if [ 条件判断式 ]
    then
        程序
fi
```

单分支，判断表达式中的空格不能省，语句的结尾为fi，如下范例

```
#!/bin/bash
# file name: if.sh
# if to go
if [ -n $1 ]
then
    cat $1
fi
```

执行结果

```
[root@theon shell_class]# sh if.sh /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:./sbin/nologin
systemd-network:x:192:192:systemd Network Management:./sbin/nologin
dbus:x:81:81:System message bus:./sbin/nologin
polkitd:x:999:998:User for polkitd:./sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
postfix:x:89:89:/var/spool/postfix:/sbin/nologin
```

双分支if语句

2、双分支if条件语句

```
if [ 条件判断式 ]
then
    条件成立时，执行的程序
else
    条件不成立时，执行的另一个程序
fi
```

范例

```
#!/bin/bash
# file name: if.sh
# if to go
if [ -n $1 ]
then
    cat $1
else
    echo "没有文件传入"
fi
```

多分支语句

```
if [ 条件判断式1 ]
then
    当条件判断式1成立时，执行程序1
elif [ 条件判断式2 ]
then
    当条件判断式2成立时，执行程序2
...省略更多条件...
else
    当所有条件都不成立时，最后执行此程序
fi
```

范例

```
#!/bin/bash
# file name: if.sh
# if to go
if [ -z $1 ]
then
    echo "请勿传入空信息"
elif [ -f $1 ]
then
    echo "文件内容为"
    cat $1
elif [ -d $1 ]
then
    echo "目录无法读取"
else
    echo "传入的文件或目录路径异常"
fi
```

扩展：数值对比的判断表达式

```
#!/bin/bash
# no demo
a=36
if (( ${a} >= 35 ))
then
    echo "a 比 35 大"
fi
```

- **case语句**

case语句和if...elif...else语句一样都是多分支条件语句，不过和if多分支条件语句不同的是，case语句只能判断一种条件关系，而if语句可以判断多种条件关系。

```
case $变量名 in
    "值1")
        如果变量的值等于值1，则执行程序1
        ;;
    "值2")
        如果变量的值等于值2，则执行程序2
        ;;
    ...省略其他分支...
    *)
        如果变量的值都不是以上的值，则执行此程序
        ;;
esac
```

范例：

```
#!/bin/bash
# file: case.sh
read -p "请选择yes/no: " -t 30 cho
case ${cho} in
    "yes")
        echo "你选择的是yes"
        ;;
    "no")
        echo "你选择的是no"
        ;;
    *)
        echo "请不要做其他选择"
        ;;
esac
```

(2.2) 循环语句

2.2.1 for循环

- for循环语法一

有几个值，循环几次，每次循环把值赋给变量执行

for 变量 in 值1 值2 值3...
do
 程序
done

范例：批量压缩文件

```
#!/bin/bash
#zip file
#v0.0.1
#filename: for_1.sh

for i in $(ls /home)
do
    echo ${i}
done
```

- for循环语法二

语法框架和一没有差别，至少循环的条件表达式不一样

for ((初始值;循环控制条件;变量变化))
do
 程序
done

范例1：从1加到100

```
#!/bin/bash
# 1 sum 100
# filename: for_2.sh
sum=0
for ((i=1;i<=100;i=i+1))
do
    sum=$(( ${sum} + ${i} ))
done
echo "和为${sum}"
```

2.2.2 while循环

while循环是不定循环，也称作条件循环只要条件判断式成立，循环就会一直继续，直到条件判断式不成立，循环才会停止。这就和for的固定循环不太一样了。

```
while [ 条件判断式 ]  
do  
    程序  
done
```

范例：while循环版本的从1加到100

```
#!/bin/bash  
# 1 sum 100  
# filename: while.sh  
num=1  
sum=0  
while (( ${num} <= 100 ))  
do  
    sum=$(( ${sum}+${num} ))  
    num=$(( ${num}+1 ))  
done  
echo "和为${sum}"
```

三、shell的函数

函数：把一个功能封装起来。使用时直接调用函数名。

这样的脚本好处：模块化，代码可读性强。

(3.1) 函数声明

函数名 ()

{

命令序列

}

或:

function 函数名() # function 可以不写

{

命令序列

}

范例

```
#!/bin/bash
sum_num()
{
    sum=${sum}+$1
}
```

需要注意: shell中的函数参数传入和其他编程语言 (go、python、C、java、php) 不一样, 不能在声明函数的时候传入参数, 比如 `def func(num)`, 只能通过全局变量或位置参数的形式传入

(3.2) 函数调用

调用语法:

函数名 参数1 参数2 ...

函数中的变量

均为全局变量, 没有局部变量

函数的调用

可以传递参数。在函数中用1、2...来引用传递的参数

范例

```
#!/bin/bash
# filename:weizhi.sh
# 定义一个函数, 接收两个参数
greet_user() {
    local name="$1" # 第一个参数
```

```

local age="$2"    # 第二个参数

echo "你好，$name!"
echo "你的年龄是 $age 岁。"
echo "函数接收到的参数总数: $#"
```

```

echo "所有参数 (\$@): \$@"
echo "所有参数 (\$*): \$*"
}

# 调用函数并传递参数
echo "--- 第一次调用 ---"
greet_user "张三" 30

echo ""

echo "--- 第二次调用 ---"
greet_user "李四" 25 "男性" # 传递了额外的参数

```

```

[root@theon shell_class]# sh weizhi.sh
--- 第一次调用 ---
你好，张三！
你的年龄是 30 岁。
函数接收到的参数总数：2
所有参数 ($@)：张三 30
所有参数 ($*)：张三 30

--- 第二次调用 ---
你好，李四！
你的年龄是 25 岁。
函数接收到的参数总数：3
所有参数 ($@)：李四 25 男性
所有参数 ($*)：李四 25 男性

```

全局变量调用方式

```

#!/bin/bash
# filename: global.sh
# 定义全局变量
GLOBAL_NAME="王五"
GLOBAL_CITY="北京"

# 定义一个函数，访问全局变量
display_global_info() {
    echo "全局名称: $GLOBAL_NAME"
    echo "全局城市: $GLOBAL_CITY"
}

# 调用函数
echo "--- 访问全局变量 ---"
display_global_info

```

四、shell的数组

在 Shell 中，数组是一种非常重要的数据结构，用于存储一系列值。Shell 提供了两种主要的数组类型

(4.1) 索引数组

索引数组是最常见的数组类型，它的元素是按照数字顺序存储的-----类似python中的list

4.1.1 数组定义

方式一：逐个赋值

```
#!/bin/bash
# 定义一个索引数组
my_array[0]="apple"
my_array[1]="banana"
my_array[2]="cherry"
```

方式二：一次性赋值 (推荐)

```
#!/bin/bash
# 定义一个索引数组
fruits=("apple" "banana" "cherry" "date" "elderberry")
```

4.1.2 访问数组元素

- 访问特定元素: `${array_name[index]}`
- 访问所有元素: `${array_name[@]}` 或 `${array_name[*]}`

`${array_name[@]}` 或 `${array_name[*]}` (两者在多数情况下相同，但
`"${array_name[@]}"` 在双引号中会保留每个元素的独立性，推荐使用)

- 获取数组长度 (元素个数): `${#array_name[@]}` 或 `${#array_name[*]}`
- 获取特定元素的长度 (字符串长度): `${#array_name[index]}`
- 获取数组的所有索引: `${!array_name[@]}` 或 `${!array_name[*]}`

示例:

```
#!/bin/bash
# filename: list.sh
fruits=("apple" "banana" "cherry" "date" "elderberry")
echo "--- 访问索引数组元素 ---"
# 访问第一个元素 (索引为 0)
echo "第一个水果: ${fruits[0]}"
# 访问第三个元素 (索引为 2)
echo "第三个水果: ${fruits[2]}"
echo ""
# 访问所有元素 (每个元素独立)
echo "所有水果 (独立):"
for fruit in "${fruits[@]"; do
    echo "- $fruit"
done
echo ""
# 访问所有元素 (作为一个整体字符串)
echo "所有水果 (一个字符串): ${fruits[*]}"
echo ""
# 获取数组长度
```

```
echo "水果数组的长度: ${#fruits[@]}"
# 获取特定元素的长度
echo "第一个水果的长度: ${#fruits[0]}" # "apple" 的长度是 5
echo ""
# 获取所有索引
echo "水果数组的所有索引: ${!fruits[@]}"
```

4.1.3 修改/添加/删除索引数组元素

- **修改元素**: 直接赋值给相应索引
- **添加元素**: 赋值给新的索引或使用 `+=`
- **删除元素**: `unset array_name[index]`
- **清空数组**: `unset array_name`

示例:

```
#!/bin/bash
#filename: change_list.sh
fruits=("apple" "banana" "cherry")

echo "--- 修改/添加/删除索引数组元素 ---"
echo "原始数组: ${fruits[@]}"

# 修改元素
fruits[1]="grape"
echo "修改索引 1 后: ${fruits[@]}"

# 添加新元素 (到下一个可用索引)
fruits[3]="kiwi"
echo "添加索引 3 后: ${fruits[@]}"

# 也可以使用 += 添加元素 (会自动追加到末尾)
fruits+=("lemon" "mango")
echo "使用 += 添加后: ${fruits[@]}"

echo "当前的数组长度: ${#fruits[@]}"
echo "当前的数组索引: ${!fruits[@]}"

# 删除元素
unset fruits[2] # 删除 "cherry"
echo "删除索引 2 后: ${fruits[@]}"
echo "删除后数组长度: ${#fruits[@]}"
echo "删除后数组索引: ${!fruits[@]}" # 注意索引 2 不再存在, 索引可能不连续

# 清空整个数组
unset fruits
echo "清空整个数组后:"
echo "数组内容: ${fruits[@]}"
echo "数组长度: ${#fruits[@]}"
```

(4.2) 关联数组

关联数组使用字符串作为键，而不是数字索引。它们在 Bash 4.0 及更高版本中可用。-----类似python的 dict

4.2.1 数组定义

需要注意： 在使用关联数组之前，你必须使用 `declare -A` 命令来声明它。

```
#!/bin/bash
# 声明一个关联数组
declare -A user_info
# 一次性赋值
# declare -A array_name=( ["key1"]="value1" ["key2"]="value2" ... )
# 定义关联数组元素
user_info["name"]="Alice"
user_info["age"]="30"
user_info["city"]="New York"
user_info["email"]="alice@example.com"
```

4.2.2 访问数组元素

- 访问特定元素: `${array_name[key]}`
- 访问所有值: `${array_name[@]}` 或 `${array_name[*]}`
- 获取数组长度 (元素个数): `${#array_name[@]}` 或 `${#array_name[*]}`
- 获取特定值的长度 (字符串长度): `${#array_name[key]}`
- 获取数组的所有键 (key): `${!array_name[@]}` 或 `${!array_name[*]}`

范例：

```
#!/bin/bash
# filename: dict.sh
declare -A user_info

user_info["name"]="Alice"
user_info["age"]="30"
user_info["city"]="New York"
user_info["email"]="alice@example.com"

echo "--- 访问关联数组元素 ---"

# 访问特定键的值
echo "用户姓名: ${user_info["name"]}"
echo "用户年龄: ${user_info["age"]}"

echo ""

# 访问所有值
echo "所有值 (独立):"
for value in "${user_info[@]"; do
    echo "- $value"
done

echo ""
```

```

# 获取数组长度
echo "关联数组的长度: ${#user_info[@]}"

# 获取特定值的长度
echo "姓名值的长度: ${#user_info["name"]}" # "Alice" 的长度是 5

echo ""

# 获取所有键
echo "关联数组的所有键: ${!user_info[@]}"

echo ""

# 遍历键值对
echo "遍历键值对:"
for key in "${!user_info[@]"; do
    echo "$key: ${user_info[$key]}"
done

```

4.2.3 修改/添加/删除关联数组元素

- **修改元素**: 直接赋值给相应键
- **添加元素**: 赋值给新的键
- **删除元素**: `unset array_name[key]`
- **清空数组**: `unset array_name`

```

#!/bin/bash

declare -A config

config["database"]="mydb"
config["port"]="5432"
config["user"]="admin"

echo "--- 修改/添加/删除关联数组元素 ---"
echo "原始配置: ${config[@]}"
echo "原始键: ${!config[@]}"

# 修改元素
config["port"]="8080"
echo "修改 port 后: ${config[@]}"

# 添加新元素
config["host"]="localhost"
echo "添加 host 后: ${config[@]}"

# 删除元素
unset config["user"]
echo "删除 user 后: ${config[@]}"
echo "删除后键: ${!config[@]}"

# 清空整个数组
unset config
echo "清空整个数组后:"

```

```
echo "数组内容: ${config[@]}"
echo "数组键: ${!config[@]}"
```

五、基于tcl编程的expect语言

我们使用shell脚本时，可能会遇到需要交互输入的情况（远程ssh连接、输入密码、输入配置信息等）

expect是一种类似shell的，基于tcl的语言。用于编写自动处理输入输出的（如交互式登录）脚本。

原理：expect语言，通过spawn加壳（加壳以便可以调用expect中的命令）后，通过expect命令获取交互时的字符，在通过send命令发送要执行的指令/命令。实现自动处理交互式登录... ..等的脚本。

(5.1) expect的安装

```
# rpm安装
yum -y install expect
```

源码安装

```
tar zxvf tcl8.6.4-src.tar.gz
cd tcl8.6.4/unix/
./configure --prefix=/usr/local/tcl --enable-shared
make
make install
cp tclUnixPort.h /usr/local/tcl/generic/
tar zxvf expect5.45.tar.gz
cd expect5.45
./configure --prefix=/usr/local/expect --with-tcl=/usr/local/tcl/lib --with-tclinclude=../tcl8.6.4/generic/
make
make install
ln -s /usr/local/tcl/bin/expect /usr/expect/bin/expect
echo 'export PATH=$PATH:/usr/expect/bin' >>/etc/profile
```

(5.2) expect的使用

5.1.1 定义脚本执行的环境

```
#!/usr/bin/expect
```

这里定义的是expect可执行文件的链接路径（或真实路径），功能类似于#!/bin/bash

5.1.2 超时时间

```
set timeout 30
#设置超时时间，单位是秒，如果设为timeout -1 意为永不超时
```

设置超时时间，单位是秒，如果设为timeout -1 意为永不超时

5.1.3 spawn

spawn 是进入expect环境后才能执行的内部命令，如果没有装expect或者直接在默认的SHELL下执行是找不到spawn命令的。不能直接在默认的shell环境中进行执行主要功能，它主要的功能是给ssh运行进程加个壳，用来传递交互指令。

5.1.4 expect

这里的expect同样是expect的内部命令

主要功能：判断输出结果是否包含某项字符串，没有则立即返回，否则就等待一段时间后返回，等待时间通过timeout进行设置

5.1.5 send

执行交互动作，将交互要执行的动作进行输入给交互指令

命令字符串结尾要加上"\r"，如果出现异常等待的状态可以进行核查

5.1.6 exp_continue

继续执行接下来的交互操作

5.1.7 interact

执行完后保持交互状态，把控制权交给控制台；如果不加这一项，交互完成会自动退出

5.1.8 \$argv

expect 脚本可以接受从bash传递过来的参数，可以使用 [lindex \$argv n]获得，n从0开始，分别表示第一个，第二个，第三个... ..位置参数

(5.3) expect脚本范例

编写实现自动登录ssh后，再自动添加用户的脚本

简单版：

```
#!/usr/bin/expect
#↑声明用expect环境解析
#filename: ssh_login_ex.exp
#run:expect ssh_login_ex.exp
set ip_addr 127.0.0.1
#↑声明变量
set name root
set passwd root
spawn ssh $name@$ip_addr
#↑给ssh命令加壳,以便调用接下来的expect和send命令
expect {
    "yes/no" {send "yes\r";exp_continue}
    "password:" {send "$passwd\r"}
eof
    "password:" {send "$passwd\r"}
}
#↑expect获取交互界面的字符(交互界面获取字符时expect要用{}，当获取yes/no字符时，发送yes字符并回车(\r)，然后继续，当获取password字符时，输入密码变量$passwd并回车，eof表示结束)
#↑exp_continue:继续下一条交互操作，不加则会退出
```

```
expect "*from*"
#↑expect获取字符，无交互界面时，expect不用{}
send "useradd -s /sbin/nologin cj\r"
#↑expect获取字符*from*后，就发送创建用户命令
expect "]"#
expect eof
#↑expect获取字符 ]# 后，就退出expect，必须要定义expect的退出，否则上一条send命令不会发送就会退出。
```

升级版:ssh连接大量服务器（超过100台）时

需要shell和exp脚本相互配合

expect脚本如下：

```
#!/usr/bin/expect
set ip_addr [lindex $argv 0]
set passwd [lindex $argv 1]
#设置变量，并且以位置变量的形式传递参数。 [lindex $argv 0]相当于$1,[lindex $argv 1]相当于$2。expect脚本后面应该加上2个位置变量如： ./xxx.exp ip name
spawn ssh root@$ip_addr
expect {
"yes/no" {send "yes\r";exp_continue}
"password:" {send "$passwd\r"}
eof
"password" {send "$passwd\r"}
}

expect "*from*"
send "useradd cj\r"
expect "]"#
expect eof
```

然后编写shell脚本来调用expect脚本实现在多台服务上(大于100台)自动创建用户。首先有一个写有所有远程服务器ip地址和密码的文本文件。mima.txt 假设用户都是root

mima.txt如下：

```
[root@localhost ~]# head -30 mima.txt
192.168.0.100 abc100
192.168.0.101 abc101
192.168.0.102 abc102
192.168.0.103 abc103
192.168.0.104 abc104
192.168.0.105 abc105
192.168.0.106 abc106
192.168.0.107 abc107
192.168.0.108 abc108
192.168.0.109 abc109
192.168.0.110 abc110
192.168.0.111 abc111
192.168.0.112 abc112
192.168.0.113 abc113
192.168.0.114 abc114
192.168.0.115 abc115
192.168.0.116 abc116
192.168.0.117 abc117
192.168.0.118 abc118
192.168.0.119 abc119
192.168.0.120 abc120
192.168.0.121 abc121
192.168.0.122 abc122
192.168.0.123 abc123
192.168.0.124 abc124
192.168.0.125 abc125
192.168.0.126 abc126
192.168.0.127 abc127
192.168.0.128 abc128
192.168.0.129 abc129
```

shell脚本如下:

```
#!/bin/bash
for i in $(awk '{print $1}' mima.txt)
do
j=$(awk -v I="$i" '{if(I==$1) print $2}' mima.txt)
expect /root/sh/usr_add.exp $i $j
#向expect脚本传递了2个位置参数$i和$j
done
```

其他



IT入门基础 | 网络安全运维基础知识

Theon的小猫助手

单集更新

节目详情



Shell与Expect全自动运维

介绍了 Shell 脚本编程的核心技术。并且通过Shell 脚本的输入（如何让脚本接收数据）、流程控制（if 判断、for/while 循环等逻辑）、函数与数组（...

18分钟 • 12分钟前

0 0 0

播客标题：Shell与Expect全自动运维

播客内容：深入探讨 Shell 脚本编程的核心技术

播客地址：

- 喜马拉雅：<https://www.ximalaya.com/album/118679212>
- 小宇宙：<https://namecard.xiaoyuzhoufm.com/rjrtv>

目的：辅助掌握今天的知识内容，没有时间学习的同学或者已经有基础的同学可以通过播客快速掌握大致内容

作业

情景任务：日志文件的备份

场景描述：某个企业的一个服务器上日志需要按照大小进行备份：将大于30M的日志文件进行打包备份到/var/backup目录下，并且备份后的文件名格式为 `源文件名-备份时间.tar.gz`，并且备份时间格式为年月日，比如20250229（年月日，date +%Y%m%d）

任务内容：

- 将文件secure_anaylist.zip上传到你的linux服务器上，并解压作为情景中需要背景的日志文件
- 编写shell将解压后目录secure_anaylist中的日志文件按照情景要求进行备份

任务提交：脚本执行前后的结果，参考范例

```
# 先执行 ls /var/backup
# 然后执行 脚本
# 最后执行 ls /var/backup 查看是否生成了备份文件，并且大小和文件名格式是否正确
```

```
[root@theon secure_anaylist]# ls /var/backup
ls: cannot access /var/backup: No such file or directory
[root@theon secure_anaylist]# sh work_20250529.sh
./secure-202403281711568641
./secure-202407241721763961
./secure-202409101725910501
备份完成
备份文件位于 /var/backup
[root@theon secure_anaylist]# ls /var/backup
secure-202403281711568641-250529.tar.gz  secure-202407241721763961-250529.tar.gz  secure-202409101725910501-250529.tar.gz
[root@theon secure_anaylist]#
```

参考范例